

HelixCluster

HelixCluster

A distributed operating system for AI compute — from datacenter GPUs to edge handhelds, under one control plane.

Summary

Helix Cluster OS is a next-generation distributed operating system that orchestrates compute across heterogeneous nodes — from datacenter GPUs down to edge SBCs and handhelds — unifying HPC scheduling, container orchestration, AI/ML inference, federated multi-cluster operation, and secure multi-tenant sessions under a single control plane.

Short description

A Go-based distributed OS / GPU-sharing compute cluster. It unifies HPC scheduling (an Omega-model two-level scheduler), container orchestration, AI inference routing, federation, and secure multi-tenant sessions across heterogeneous nodes, coordinated by SWIM gossip and Raft consensus, with post-quantum end-to-end encryption.

Long description

Helix Cluster OS orchestrates compute workloads across radically heterogeneous hardware — datacenter GPUs, edge single-board computers, even handhelds — under one control plane, treating a rack of A100s and a fistful of SBCs as one addressable fabric instead of a dozen incompatible islands. It is a Go workspace (a monorepo plus git submodules) implementing a seven-layer stack, from the L0 hardware substrate up through L7 federation and observability, coordinated by fourteen control-plane microservices. Node membership is tracked with SWIM gossip and discovery so the fabric self-heals as nodes join and leave;

strongly-consistent state rides on Raft consensus, organised as per-shard Raft groups with leaseholder-local reads for speed and STONITH fencing to guarantee a partitioned node can't corrupt shared state. Workload placement runs through an Omega-model two-level scheduler — optimistic concurrency, ClassAd matching, gang scheduling, value-multiplier preemption, and constraint-based placement — then goes further than a classic HPC scheduler ever did: carbon-aware and cost/TCO-aware routing, burst-to-cloud autoscaling, and marketplace adapters (Akash, io.net, RunPod, AWS Spot, Chutes) that let a job spill onto rented capacity when local supply runs dry.

End users don't see any of that machinery directly; they interact through a clean session model (compute allocations), an interactive WebSocket/PTY terminal, an internal AI-inference route, and pool-utilization reads. Security is a first-class layer rather than a bolt-on: SPIFFE identity, device attestation (challenge/response, proof-of-GPU-work, sealing), an export-control KYC gate, and a post-quantum end-to-end-encrypted transport built on an X25519 + ML-KEM-768 hybrid key exchange with AEAD record protection and replay rejection — engineered so today's captured traffic stays confidential even against tomorrow's quantum adversary. Correctness is not asserted, it is *demonstrated*: deterministic simulation testing (FoundationDB-style seeded runs, fault injection, network simulation, byte-for-byte replay, and a Porcupine linearizability checker) reproduces distributed failures on demand, and mandatory paired mutation testing proves the guard tests actually bite. Architecture and docs are kept honest by mechanical lints that fail the build the moment reality and documentation drift apart.

Why we built it

To run AI and HPC workloads across wildly different hardware tiers without stitching together separate schedulers, orchestrators, and inference stacks — and to do it with an engineering guarantee that every shipped feature proves *real end-user behaviour* (never green tests over stubs) and every OS-specific capability uses a real native facility per platform (no Linux-only mocks). The motivating problem, quoted in the repo's governance, is the “tests pass but the feature doesn't actually work” failure mode, which the project is explicitly built to eliminate.

Why it's a game-changer

It collapses five things that are normally five separate stacks — HPC scheduling, container orchestration, AI inference, multi-cluster federation, and secure multi-tenant sessions — into a

single control plane that stretches from datacenter GPUs all the way down to edge handhelds. And it does so with a rigor budget usually reserved for specialised infrastructure: formal-methods-grade correctness (TLA+ specs, deterministic simulation, linearizability checking) and post-quantum confidential transport, the kind of guarantees most orchestrators simply don't attempt. On top of the technical differentiation, cost- and carbon-aware placement plus cloud-marketplace burst make it an *economic* lever too — the scheduler can chase cheaper, greener, or spare capacity automatically, so the same workload costs less and emits less without anyone rewriting a job.

What's innovative

- **Deterministic Simulation Testing (DST)** — a seeded, fully reproducible simulator that injects faults, clock skew, and network partitions, replays them byte-for-byte, and runs the result through a Porcupine linearizability checker, so a Heisenbug caught once can be reproduced on command forever.
- **Omega-model two-level scheduler** — optimistic-concurrency placement with ClassAd matching, gang scheduling, and value-multiplier preemption, the shared-state design that lets many schedulers commit against one cluster without a central bottleneck.
- **Post-quantum E2EE / confidential inference wiring** — an X25519 + ML-KEM-768 hybrid key exchange with per-request response-keypair binding and AEAD with replay rejection (the crypto primitives are real and tested; the full confidential multi-node round-trip remains explicitly PLANNED/gated).
- **Attestation-driven trust** — nodes must *prove* what they are: SPIFFE identity, proof-of-GPU-work, device sealing, an export-control KYC gate, and EU AI Act compliance doc-gen, so trust is earned by evidence rather than assumed by network position.
- **Cost- and carbon-aware orchestration** — TCO modeling, carbon-aware placement, burst-to-cloud, an N+K failover reserve, and cloud-marketplace adapters, making price and emissions first-class scheduling inputs instead of afterthoughts.
- **Multi-Raft consensus** — per-shard Raft groups with leaseholder-local reads for low-latency consistency, backed by STONITH fencing (IPMI / EC2 / Azure / SBD) so a wedged node is decisively removed, not left to corrupt state.
- **Anti-drift mechanical lints** — archlint fails the build the instant a documented component maps to a package path that doesn't exist, and a docs-chain engine keeps Markdown / HTML / PDF / DOCX byte-consistent, so the documentation can't quietly lie about the code.

Biggest technical challenges & how we solved them

- **“PASS-bluff” (tests pass on non-functional features).** The failure mode the whole project is built to kill: a green suite over stubs. Solved with mandatory paired mutation testing — every work item carries a named guard test that must *fail* under an independent code mutation before the item can be marked complete, so a passing test provably exercises real behaviour rather than a mock.
- **Cross-platform parity (no Linux-only mocks).** Solved with one shared interface split by build tags onto genuine per-OS facilities — Linux cgroup / /proc / kernel WireGuard, macOS sysctl / vm_stat / IOKit / wireguard-go — and then cross-checked against an independent OS oracle, so each platform reports true native state instead of a Linux fiction.
- **Distributed correctness under faults.** Solved with deterministic simulation testing and a linearizability checker that manufacture and replay partitions, crashes, and clock skew, backed by TLA+ formal specs that pin down the consensus and scheduling invariants before a line of code runs.
- **Documentation and architecture drift.** Solved by archlint, which fails the build on any documented-but-nonexistent mapped package, and a docs_chain verify gate with no escape hatch — drift is a build break, not a stale wiki page.
- **Honest scoping of unfinished work.** The confidential multi-node inference round-trip is deliberately gated behind a ticket and labeled “not yet validated end-to-end” rather than dressed up as shipped — the same discipline applied to what *isn't* done yet as to what is.

Tech stack

- **Go (go.mod: 1.25 / toolchain 1.26.4)** — the control-plane language across a ~30-module workspace; chosen for cheap goroutine concurrency and static binaries that deploy identically from datacenter to edge.
- **Zig (0.14+) + C/C++** — reached for exactly where Go’s runtime gets in the way: low-level system primitives and GPU kernels that need deterministic, allocation-free control over the hardware.
- **gRPC + Protocol Buffers** — every inter-subsystem API (api/v1/) is a typed, versioned contract, so the fourteen microservices evolve without breaking each other or hand-rolling wire formats.
- **Raft (etcd-raft) + SWIM gossip** — a deliberate split: Raft carries the state that *must* be strongly consistent, while SWIM gossip handles membership and discovery at scale where consensus would be too heavy.

- **PostgreSQL 16, Redis 7 cluster, etcd v3.5, SQLite** — the right store for each job: Postgres for durable relational state, Redis for hot cache, etcd for coordination, and embedded SQLite for the node-local HXC work-item registry.
- **NATS 2.10 (JetStream), Kafka 4.0 (KRaft), RabbitMQ 3.13** — three messaging backbones for three shapes of traffic: NATS/JetStream for fast internal eventing, Kafka for durable high-throughput streams, RabbitMQ for classic broker semantics.
- **WireGuard mesh + ML-KEM-768/X25519 + AES-256-GCM/ChaCha20-Poly1305 + HKDF** — WireGuard for a lean node-to-node mesh, wrapped in a hybrid post-quantum handshake and AEAD records so the transport is confidential against both classical and quantum attackers.
- **SPIFFE + JWT (HS256) + scope-based RBAC + OPA** — layered identity and authorization: SPIFFE for workload identity, JWT for tokens, scope-based RBAC for coarse access, and OPA to express fine-grained policy as code.
- **Prometheus v2.50, Grafana 10.4, Jaeger 1.55, W3C tracing** — metrics, dashboards, and distributed traces with W3C context propagation, so a request can be followed across services and hardware tiers.
- **HashiCorp Vault 1.16** — secrets and key material kept out of code and config and issued under audit.
- **Docker Compose, Kubernetes (kustomize, hardened securityContext), Helm** — Compose for local bring-up and Kubernetes/Helm with hardened security contexts for real deployments, one definition promoted across environments.
- **React + TypeScript + Vite (Node 20+)** — a fast, type-safe web UI for sessions, terminals, and pool utilization.
- **TLA+** — formal specification of the consensus and scheduling invariants, so the hardest-to-test properties are proven at the design level before implementation.

Status & honesty notes

- **Status: in-development.** The version is early (0.1.0-dev). Several advanced features — full confidential multi-node inference round-trip, marketplace settlement, and attestation-driven scheduling population — are explicitly labeled **PLANNED** / infra-gated in the repository and are **not** presented as fully working. Coverage figures are self-reported.
- **License: TBD.** Not clearly declared; the Helm chart's HelixCluster/HelixCluster and helixcluster.io URLs are unverified placeholders that do not match the real remotes.
- The bundled LLM stack projects (LLMOrchestrator, LLMProvider, LLMsVerifier) are decoupled submodules, not model servers hosted inside the cluster.

Priority tier: Helix-primary (LLM-infrastructure cluster — the compute substrate that can host inference and compute workloads). Ranks after HelixTrack.